Python for computational Problem Solving · Unit_03 · Unit 3: Functions, Functional Programming and Modules

DRAFT — UE26CS151A · Unit_03 · Unit 3: Functions, Functional Programming and Modules · module

Standalone video lesson · authored study notes with source citations

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the topic explanation, worked examples, and recap. Evidence citations point to source transcript intervals and slide pages; full slide images remain in the source visual index. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · module
2. Appendix A · Symbol glossary
3. Appendix B · Sources, coverage and gaps
4. Appendix C · Revision and retrieval practice

M1

module

1 module

Study focus: module. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000094 · EV0000095

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Today we are going to explore one of the most powerful features of Python, which is modules.

Key details

- Modules help us write a clean, organized and reusable code and then play a major role in real-world software development.
- A module is simply a Python file that contains code functions, classes, or variables.
- That we can reuse them in other programs.
- Python already gives us many built-in modules like math.
- OS, random and date and time, etc.
- That makes our life easier.
- And the best part is we can create our own modules.
- We can create our own modules and even we have third party modules.
- What are the advantages of modules?
- Why do we use modules?
- First one is reusability.
- You write the code once and use it again and again.
- This saves a huge amount of time.
- Instead of writing code one, writing one long program, we break into smaller parts.
- Each part becomes easier to develop and debug.
- Next comes separate namespaces.
- Every module has its own space for names.
- Better organization, modul us structure a project neatly, especially when it grows larger.
- In short, modules makes our programs cleaner, smarter, and scalable.
- Next comes types of modules.

- We have three types built-in modules, user defined modules, and third party modules.
- Pip is a command to install packages.
- This is a tool to install and manage the third party Python packages.
- Then if you want to uninstall the module, you can give pip uninstall and the module name.
- And if you want to see what are the modules that you have installed.
- PIP list gives you a view of what all modules that you have installed.
- Almost every advanced Python project uses PIP.
- Next one is.
- We'll talk about the types of modules.
- Built-in modules.
- Built-in modules come packaged with Python.
- You don't need to install anything, just import them and use them.
- These modules for everyday programming needs.
- Next comes built-in modules, ways to import the built-in modules.
- and the module name.
- Import specific attributes.
- From math import square root.
- It is enough if you use square root, the just the function name.
- And in this case,.
- When you are importing specific attributes, you can only use those specific ones which you have imported, not all of them.
- Importing all from math import start.
- This is not recommended.
- If you don't if the specific attributes or specific functions, please import only them, not import star.
- Then next one is user defined modules.
- Whenever we write a Python file with some useful functions and save it as.py file, that can be imported to another program.
- This is an excellent way to organize your own function and reuse them across projects.

Example or application from the lecture

- Let's look at the advantages.
- Let's go deeper into each of these types.
- Before that, let's see what is pip.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000094 · EV0000095 · EV0000096 · +1 source units in pack.json

Core explanation · 05:00–09:59

Example files that you may create is you can create a.py file of any program and just you need to import use the inp keyword and import the program.

Key details

- It is the same like the built-in modules, but here you have created the own PY5.

- Then comes third party modules.
- Finally we have third party modules.
- These are created by Python community and gives us advanced capabilities.
- Some of the third party modules are listed down here.
- We have NumPy for numerical computing and we have Pandas for data analysis.
- or working with data sets.
- We have mapplotlib for charts and graphs, basically plotting and visualization.
- We have requests for fetching data from websites or working with APIs.
- We have pillow for image processing or image editing.
- These tools expand Python beyond basic programming and enable the real world application.

Example or application from the lecture

- Okay, example like NumPai, Skycatlone, Pillow, all of them has advanced capabilities.
- Built-in modules gives us ready-made functionality, user defined modules let us organize our own code, and third party modules bring the power of the entire Python community right into our projects.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000098 · EV0000099 · EV0000100 · +1 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
module transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
module transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
module transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
module transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_03/AV_Summaries/013_module__3e602706ded0701e.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
2. 4 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
3. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
4. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
5. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
6. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00034	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...
SRC00035	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...
SRC00036	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00037	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0029. Sidecar variants compare...

C Revision and retrieval practice